

PHASE 4: FINAL MODEL TRAINING - STEP-BY-STEP INSTRUCTIONS

Prepared by: Christos (via Claude Code analysis of 200-repeat nested CV results)

Date: 2026-05-01

Winner: EN+NMC plateau ensemble (en_nmc_pens), BA2 0.759

This document explains exactly how to produce the four submission deliverables:

1. model/model.pkl - Serialised final model
2. model/run_model.py - Prediction script
3. results/prediction.txt - 57 validation predictions
4. results/estimate.txt - Accuracy estimate (single number, 0-57)

BACKGROUND: WHAT THE FINAL MODEL DOES

The model is a two-stage hierarchical classifier:

Stage 1: KW+RF (k5) - classifies HER2+ vs rest (binary)

- Kruskal-Wallis selects top 5 features (always chr17q ERBB2 amplicon)
- Random Forest with fixed hyperparameters (see below)
- Threshold 0.5 on predict_proba
- Samples with $P(\text{HER2+}) > 0.5$ are predicted HER2+
- Remaining samples are routed to Stage 2

Stage 2: EN+NMC plateau ensemble (15 models) - classifies HR+ vs TN

- 15 pre-determined hyperparameter configurations (listed below)
- Each config: StandardScaler -> ElasticNetSelector -> NearestCentroidWithProba
- All 15 models are trained on the SAME data (all non-HER2+ training samples)
- Predictions: average predict_proba across 15 models, then argmax

For final training, fit Stage 1 on ALL 100 samples and Stage 2 on ALL 68 non-HER2+ samples (36 HR+ + 32 Triple Neg).

PREREQUISITES

1. Activate the conda environment:

```
source ~/miniconda3/bin/activate tb_310
```

2. Ensure these files exist:

data/Train_call.tsv - 100 training samples (raw CN calls)
data/Train_clinical.tsv - 100 sample labels
data/Validation_call.tsv - 57 unlabelled validation samples

3. Ensure preprocessing has been run (merged data exists):

results/2026-04-25_final_hierarchical/preprocessing/data/train_merged.tsv

If not, run:

```
python3 code/preprocessing_phase.py --name final_hierarchical
```

4. Key code files you will need to import from:

code/utis/cv_components.py - KruskalWallisSelector, ElasticNetSelector,
NearestCentroidWithProba
code/utis/cv_config.py - build_pipeline, build_stage2_pipeline
code/utis/cv_io.py - load_cv_data
code/utis/constants.py - MAX_PLATEAU_SIZE, class constants
code/utis/paths.py - _find_or_create_run_dir

STAGE 1 RF HYPERPARAMETERS (FIXED)

Selected by code/select_stage1_params.py (10 repeats x 5 folds, 54 RF configs).

All configs with max_featuresNone achieve perfect BA1.0 and margin1.0.

```
n_estimators: 500  
max_depth: None (unlimited)  
min_samples_leaf: 1  
max_features: None (use all 5 features per tree - important!)  
class_weight: "balanced"  
random_state: 42
```

NOTE: The current build_pipeline("kw_rf") uses sklearn's default max_features ("sqrt"), which means $\sqrt{5}$ features per tree. For the final model, override with max_featuresNone for maximum robustness. Set k5 with:
stage1_pipe.set_params(selector__k5, clf__max_featuresNone)

STAGE 2 PLATEAU ENSEMBLE: 15 HYPERPARAMETER CONFIGURATIONS

These 15 configs were identified by pooling inner CV scores across all 1000 nested CV folds (200 repeats x 5 outer folds). They represent the "plateau" of near-optimal hyperparameter combinations (best_mean - 1*std threshold).

All 15 use selector__top_k50. The varying params are:

- selector__C (ElasticNet regularisation strength)
- selector__l1_ratio (L1 vs L2 mixing)
- clf__shrink_threshold (NMC shrinkage; None no shrinkage)

Rank	Score	shrink_threshold	C	l1_ratio	top_k
1	0.7432	None	0.0464159	0.1	50
2	0.7428	0.2	0.0464159	0.1	50
3	0.7426	0.1	0.0464159	0.1	50
4	0.7417	0.2	0.16681	0.5	50
5	0.7411	0.1	0.16681	0.5	50
6	0.7391	0.5	0.16681	0.5	50
7	0.7388	None	0.16681	0.5	50
8	0.7388	0.5	0.0464159	0.1	50
9	0.7387	0.2	0.599484	0.7	50
10	0.7386	0.1	0.16681	0.3	50
11	0.7386	0.2	0.16681	0.3	50
12	0.7385	None	0.16681	0.1	50
13	0.7380	0.2	0.16681	0.1	50
14	0.7376	0.1	0.16681	0.1	50
15	0.7376	0.1	0.599484	0.7	50

WHAT THE FINAL TRAINING SCRIPT NEEDS TO DO

The script (code/final_training_phase.py) should:

1. LOAD DATA

- Load train_merged.tsv and Train_clinical.tsv via load_cv_data()
- Load Validation_call.tsv for prediction
- Apply the SAME merging as training data to validation data
(use preprocessing_phase.py's merge logic, or load the merge map)

2. FIT STAGE 1

- Create binary labels: y_s1 (y HER2+_index).astype(int)
- Build KW+RF pipeline: build_pipeline("kw_rf", random_state42)
- Set params: selector__k5, clf__max_featuresNone

- Fit on ALL 100 samples with binary labels
- Apply to validation: predict_proba, threshold at 0.5

3. FIT STAGE 2 (15 models)

- Identify non-HER2+ training samples (68 samples)
- Create binary labels for Stage 2: HR+0, TN1 (or use LabelEncoder)
- For each of the 15 plateau configs:
 - a. Build pipeline: build_stage2_pipeline("en_nmc", random_state42)
 - b. Set params from the table above
 - c. Fit on all 68 non-HER2+ training samples
 - d. Store the fitted pipeline
- For validation samples routed to Stage 2:
 - a. Run predict_proba on each of the 15 fitted pipelines
 - b. Average the 15 probability matrices
 - c. Argmax -> predicted class (HR+ or TN)

4. COMBINE PREDICTIONS

- Samples predicted HER2+ by Stage 1 -> "HER2+"
- Remaining samples -> Stage 2 prediction ("HR+" or "Triple Neg")

5. SAVE OUTPUTS

- model/model.pkl: joblib.dump of a dict containing:


```
{
  "stage1_pipeline": fitted_stage1_pipe,
  "stage1_threshold": 0.5,
  "stage2_pipelines": [list of 15 fitted pipelines],
  "label_encoder": le,
  "feature_names": feature_names,
  "merge_map": <however you handle validation merging>,
}
```
- results/prediction.txt: TSV with header "Sample"\t"Subgroup", 57 rows
- results/estimate.txt: single number (see Accuracy Estimate below)
- model/run_model.py: script that loads model.pkl and predicts

CRITICAL: VALIDATION DATA MERGING

The training data was merged from 2834 raw regions to 273 consensus segments ($r > 0.8$ adjacent correlation merging). The validation data (Validation_call.tsv) has the SAME 2834 raw regions but needs to be merged using the SAME merge map.

The merge map was computed during preprocessing_phase.py. You need to:

Option A: Save the merge map (which raw regions -> which merged segment) during preprocessing and re-apply it to validation data.

Option B: Load the raw training data's region boundaries, load the merge map from the preprocessing output, and apply the same grouping + median aggregation to the validation data columns.

Check how preprocessing_phase.py stores the merge mapping. The key file is: results/2026-04-25_final_hierarchical/preprocessing/data/train_merged.tsv

- This has 273 rows with Chromosome, Start, End columns
- The validation data must be collapsed to match these exact 273 segments

Look at code/preprocessing_phase.py for the merge_adjacent_regions() function. The merge map can be reconstructed: for each merged segment, identify which raw regions (by Start/End coordinates) fall within it, then take the median CN value across those raw regions for each validation sample.

model/run_model.py TEMPLATE

The prediction script must accept these arguments:

```
python3 model/run_model.py -i <input.tsv> -m <model.pkl> -o <output.txt>
```

It should:

1. Load model.pkl with joblib.load
2. Read input TSV (same format as Validation_call.tsv)
3. Apply the merge map to reduce 2834 regions -> 273 segments
4. Run Stage 1: predict HER2+ vs rest
5. Run Stage 2: average 15 models' probabilities for non-HER2+ samples
6. Write output TSV: "Sample" "Subgroup" header + 57 prediction rows

Verify output format with:

```
python3 code/predictions_file_format_check_v2.py results/prediction.txt
```

ACCURACY ESTIMATE (results/estimate.txt)

This file contains a single number between 0 and 57 (expected number of correct predictions on the 57 validation samples).

From 200-repeat nested CV:

- Combined 3-class BA (all training data): ~0.83
- Estimated correctly classified $0.83 \times 57 \sim 47$

But the plateau ensemble's true deployment BA2 is probably ~0.755 (mild leakage correction), giving combined 3-class BA ~ 0.84.

Recommended estimate: 47 (conservative) or 48 (optimistic).

Use: `echo "47" > results/estimate.txt`

VERIFICATION CHECKLIST

After training:

- [] `model/model.pkl` exists and loads with `joblib.load`
- [] `model/run_model.py` runs:


```
python3 model/run_model.py -i data/Validation_call.tsv \
  -m model/model.pkl -o output.txt
```
- [] `output.txt` matches `results/prediction.txt` exactly
- [] Format check passes:


```
python3 code/predictions_file_format_check_v2.py results/prediction.txt
```
- [] `results/estimate.txt` contains a single number (47)
- [] `prediction.txt` has 58 lines (1 header + 57 predictions)
- [] Header is exactly: "Sample"\t"Subgroup" (with quotes)
- [] All predictions are one of: HER2+, HR+, Triple Neg

FILES SUMMARY

Existing files you need:

- `code/utils/cv_components.py` - Custom sklearn components
- `code/utils/cv_config.py` - Pipeline builders
- `code/utils/cv_io.py` - Data loading
- `code/utils/constants.py` - Constants
- `code/preprocessing_phase.py` - Merging logic (reference for validation)
- `code/select_stage1_params.py` - Stage 1 param selection (already run)
- `results/2026-04-25_final_hierarchical/preprocessing/data/train_merged.tsv`

Files to create:

- `code/final_training_phase.py` - Main training script
- `model/run_model.py` - Prediction script for submission
- `model/model.pkl` - Serialised model (generated by script)
- `results/prediction.txt` - 57 predictions

results/estimate.txt

- Single number (47)